

بسم الله الرحمن الرحيم

Authentication & API

المقدمة :

تُعد واجهات برمجة التطبيقات (APIs) وأنظمة التحقق من الهوية من أهم التقنيات المستخدمة في تطوير تطبيقات الويب والموبايل الحديثة، حيث تسمح للتطبيقات المختلفة بالتواصل وتبادل البيانات بطريقة منظمة وآمنة. كما تساعد أنظمة المصادقة والتفويض في حماية بيانات المستخدمين والتحكم في الصلاحيات.

- ما هو الـ API ؟

الـ API اختصار لـ Application Programming Interface ، وهو وسيلة تسمح لتطبيقات أو نظامين بالتواصل وتبادل البيانات فيما بينهما.

مثال: عندما يستخدم تطبيق الطقس بيانات الطقس من موقع آخر، فإنه يطلب البيانات عبر الـ API لماذا يوجد؟

يوجد الـ API لحل مشكلة التواصل بين الأنظمة المختلفة بطريقة سهلة ومنظمة دون الحاجة لمعرفة تفاصيل النظام الداخلي.

ما المشكلة التي يحلها؟

بدون API سيكون من الصعب على التطبيقات المختلفة التواصل مع بعضها أو تبادل البيانات. الـ API يوفر طريقة منظمة وآمنة للتواصل بين الـ Frontend والـ Backend أو بين الأنظمة المختلفة.

كيف يتواصل الـ Frontend والـ Backend ؟

الـ Frontend يرسل Request طلب .

الـ Backend يستقبل الطلب ويعالجه.

ثم يعيد Response (استجابة) تحتوي على البيانات أو النتيجة.

أمثلة عليه :

تطبيقات الطقس.

تسجيل الدخول باستخدام Google.

تطبيقات الدفع الإلكتروني.

تطبيقات الخرائط مثل Google Maps.

Endpoints and REST --2

ما هو REST ؟

REST هو أسلوب لتنظيم بناء الـ APIs باستخدام بروتوكول HTTP .

ما هي الـ Endpoints ؟

هي الروابط التي يرسل إليها التطبيق الطلبات.

أنواع الطلبات :

GET

تستخدم لجلب البيانات.

مثال: جلب قائمة المستخدمين.

POST

تستخدم لإضافة بيانات جديدة.

مثال: إنشاء حساب جديد.

PUT

تستخدم لتعديل بيانات موجودة.

مثال: تعديل معلومات المستخدم.

DELETE

تستخدم لحذف البيانات.

مثال: حذف حساب مستخدم.

لماذا نستخدمها؟

لتنظيم العمليات المختلفة داخل الـ API بطريقة واضحة.

ما المشكلة التي تحلها؟

ينظم طريقة إرسال الطلبات والتعامل مع البيانات، حتى لا تصبح العمليات عشوائية أو غير مفهومة بين التطبيقات.

Request & Response - 3

ما هو Request ؟

هو الطلب الذي يرسله العميل (Client) إلى الخادم (Server) ويحتوي غالباً على:

الرابط URL

نوع الطلب

البيانات المطلوبة

ما هو Response ؟

هو الرد الذي يعيده الخادم بعد معالجة الطلب.

ويحتوي على:

البيانات

حالة الطلب (Status Code)

أكواد الحالة المشهورة

200

الطلب نجح.

201

تم إنشاء بيانات جديدة بنجاح.

400

خطأ في الطلب.

401

غير مصرح بالدخول.

404

الصفحة أو البيانات غير موجودة.

500

خطأ داخلي في الخادم.

لماذا توجد؟

لتوضيح نتيجة الطلب ومعرفة إن كان ناجحاً أو يحتوي على مشكلة.

أمثلة عليه:

عند تسجيل الدخول.

تحميل البيانات من السيرفر.

إرسال نموذج تسجيل.

ما المشكلة التي يحلها؟

يوضح كيف يتم إرسال واستقبال البيانات بين العميل والخادم بطريقة مفهومة ومنظمة، ويساعد في معرفة نجاح الطلب أو وجود خطأ.

Authentication vs Authorization --4

ما هو الـ **Authentication** ؟

يعني التحقق من هوية المستخدم.

مثال: التأكد من اسم المستخدم وكلمة المرور.

ما هو الـ **Authorization** ؟

يعني تحديد الصلاحيات بعد تسجيل الدخول.

مثال: هل المستخدم يستطيع حذف البيانات أم لا.

الفرق بينهما

Authentication = من أنت؟

Authorization = ماذا تستطيع أن تفعل؟

لماذا نستخدمهما؟

لحماية الأنظمة والبيانات من الوصول غير المصرح به.

أمثلة :

أنظمة الجامعات.

الحسابات البنكية.

لوحات تحكم المواقع.

ما المشكلة التي يحلها؟

يحمي النظام من دخول المستخدمين غير المصرح لهم، ويحدد صلاحيات كل مستخدم داخل النظام.

Basic Auth & Tokens --5

ما هو الـ **Basic Authentication** ؟

طريقة بسيطة لإرسال اسم المستخدم وكلمة المرور مع الطلب.

لكنها أقل أماناً إذا لم تُستخدم مع HTTPS.

ما هي الـ **Tokens** ؟

هي رموز يتم إنشاؤها بعد تسجيل الدخول وتستخدم للتحقق من المستخدم بدل إرسال كلمة المرور كل مرة.

ما هو الـ **JWT** ؟

JWT اختصار لـ JSON Web Token ، وهو نوع مشهور من الـ Tokens يحتوي على معلومات المستخدم بشكل آمن.

ما المشكلة التي يحلها؟

يوفر طريقة آمنة لتسجيل الدخول والتحقق من هوية المستخدم دون إرسال كلمة المرور في كل مرة.

كيف يعمل تسجيل الدخول؟

المستخدم يدخل البريد وكلمة المرور.
السيرفر يتحقق من البيانات.
إذا كانت صحيحة يتم إنشاء Token.
يستخدم الـ Token للوصول إلى الصفحات المحمية.

لماذا نستخدم الـ Tokens ؟

أكثر أماناً.
تقلل إرسال كلمة المرور باستمرار.
مناسبة لتطبيقات الموبايل والويب الحديثة.

أمثلة :

تطبيقات التواصل الاجتماعي.
المتاجر الإلكترونية.
تطبيقات البنوك.

6-- Password Hashing

ما هو الـ Hashing ؟

هو تحويل كلمة المرور إلى قيمة مشفرة لا يمكن قراءتها بسهولة.
مثال: بدل تخزين:

Bash

123456

يتم تخزين قيمة مشفرة مثل:

Bash

e10adc3949ba59abbe56e057f20f883e

لماذا لا نخزن كلمات المرور كنص عادي؟

لأن اختراق قاعدة البيانات سيكشف جميع كلمات المرور الحقيقية.

لماذا الـ Hashing مهم؟

حماية بيانات المستخدمين.
زيادة أمان النظام.
تقليل مخاطر الاختراق.

ما المشكلة التي يحلها؟

يحمي كلمات المرور من السرقة في حال اختراق قاعدة البيانات، لأن كلمات المرور لا تُخزن كنص عادي.

أمثلة :

مواقع التواصل الاجتماعي.

الأنظمة البنكية.

مواقع التسوق الإلكتروني.

الخاتمة:

تعتبر الـ APIs وأنظمة Authentication من الأساسيات المهمة في تطوير التطبيقات الحديثة، حيث تساعد في تبادل البيانات بشكل منظم وآمن، كما تضمن حماية المستخدمين وبياناتهم من الوصول غير المصرح به. فهم هذه المفاهيم يساعد المطورين على بناء تطبيقات قوية وآمنة وسهلة الاستخدام.